

bomblab 报告

姓名：戴铭秋

学号：2024201526

总分	phase_1	phase_2	phase_3	phase_4	phase_5	phase_6	secret_phase
7	1	1	1	1	1	1	1

scoreboard 截图：

2024201526	0	0	1	0	0	1	176	0	0	0	0	0	0	178	7
------------	---	---	---	---	---	---	-----	---	---	---	---	---	---	-----	---

解题报告

phase_1

答案：Sho fu briyu praffi stassui tsenva chies, Ien ryus sois nyat pyaro shennie fru.

首先可以看到调用了一个<strings_not_equal>，可以得知是一个字符串比较，而他的前一行是一个赋值操作，将（基地址+0x3180）处的字符串赋值，接着一个test比较，也就是先前输入的与（基地址+0x3180）处的字符串进行比较，由jne 144e <phase_1+0x19> 得知，如果不等便会跳转到炸弹爆炸处，因此我们只需去找到0x3180处的字符串然后在函数开始时输入即可

phase_2

答案：403295 618398 409883 587284

callq scanf 和 cmp \$0x4, %eax 读取输入后检查是否为4个数，接着代码包含了两层循环，外层循环(%r11d) 执行 2 次，内层循环(%r8d) 执行 2 次，总共计算 4 个值。

148e 和 14bb 加载了两个内存地址：0x6130 (matA) 和 0x6110 (matB)。这是两个静态数组。数组进行了一些加法和乘法操作，应该是两个矩阵的乘法，最终将输入的结果与乘法结果进行比较，不相等则爆炸。需要找到偏移量为 0x6130和0x6110处两个矩阵所存的值，通过矩阵乘法操作得到最终的四个数，然后在函数开始时输入即可

```
//矩阵乘法过程
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 2; j++) {
        int sum = 0;
        for (int k = 0; k < 3; k++) {

            int valA = matA[i * row_stride + k];
            int valB = matB[k * col_stride + j];
            sum += valA * valB;

        }
    }
}
```

phase_3

答案: 5 -460

callq 1150 [_isoc99_sscanf@plt](#) cmp \$0x1,%eax jle 1579 <phase_3+0x35> 检查开始的输入个数是否大于1个, 否则跳往爆炸处。

cmpl \$0x0,0x4(%rsp) cmpl \$0x7,(%rsp) 两句及其跳转条件可知 $y < 0$ $x \leq 7$ 。

在跳转得到的地址处发现 (偏移量+0x15b0处) 有一个炸弹爆炸, 所以跳转的地址应该不经过该处, 因此排除0、1、2, 而且在 cmpl \$0x5,(%rsp) 可知 $x \leq 5$, 否则会跳转到爆炸处。接着从3开始看, 跳转过去后经前面赋值的值计算发现只有 $x=5$ 时, y 的值会小于0, 得到-460, 所以得到答案, 并在函数开始时输入

```
//分支结构
switch (x) {
    case 0:
        result = 0x2dc;

    case 1:
        result -= 0x10a;
    case 2:
        result += 0x107;
    case 3:
        result -= 0x1cc;
    case 4:
        result += 0x1cc;
    case 5:
        result -= 0x1cc;
    case 6:
        result += 0x1cc;
    case 7:
        result -= 0x1cc;
        break;
    default:
        explode_bomb();
}
```

phase_4

答案: 31 BA

```
//func4_1是一个乘方递归计算
int val = func4_1(n - 1);
return val + val + 1;
```

func4_2是一个单步输出函数, 将汉诺塔的一步移动显示出来。

lea 0x10(%rsp),%rcx lea 0xc(%rsp),%rdx 这两句赋值根据大小得出一个参数应该是整数，一个参数为字符串。

mov \$0x5,%edi callq 1633 <func4_1> 是将调用函数1并以5为输入参数，得到31。

接着通过callq 1c1a <string_length> cmp \$0x2,%eax 得出字符串的长度应该是2，通过0x42、0x43、0x41、0x16、0x5对参数赋值可知func4_2的调用应为

```
func4_2(5, 20, 'B', 'C', 'A', generated_string);
```

可知 $20 > 2^4 - 1$ ，所以进入后半段中， $20 - 16 = 4$ ，而 $2^2 - 1 = 3$ ，在这之后的第一步即为所求的地方，而此步的操作为将最大的盘子由B移向A，因此字符串因为BA

phase_5

答案：mmmmmm

callq 1c1a <string_length> 和 cmp \$0x6,%eax 得出需要输入的字符串长度为6。

在偏移量为0x3260的地方得到数组值为

x3260 <array.0>:	2	10	6	1
0x3270 <array.0+16>:	12	16	9	3
0x3280 <array.0+32>:	4	7	14	5
0x3290 <array.0+48>:	11	8	15	13

由and \$0xf,%edx可知，保留字符的后四位，也就是后四位作为索引去对应数组的值。

cmp \$0x30,%ecx这句说明最后的值应该等于48，也就是从数组中找六个数使得他们相加为48，因此可选择6个8，而m所对应的为8，答案为mmmmmm

phase_6

答案：5 3 1 4 6 2

sub \$0x1,%eax 和 cmp \$0x5,%eax 得到输入的参数应该要小于等于6，也就是6个数分别为1-6的排序。

mov (%r12,%rbx,4),%eax 和 cmp %eax,0x0(%rbp) 将前数与后数进行一一比较，不能又相同的数出现。

六个节点的值存在偏移量为0x6210的地方，查看得到

0x6210 <node1>:	520	1	25120	0
0x6220 <node2>:	851	2	25136	0
0x6230 <node3>:	367	3	25152	0
0x6240 <node4>:	845	4	25168	0
0x6250 <node5>:	147	5	24928	0
0x6260 <host_table>:	13896	0	13901	

发现只有五个节点有值，但通过第五个节点可以得到第六个节点的地址，查询得到第六个节点的值为849。

```
mov    0x8(%rbx),%rax
mov    (%rax),%eax
cmp    %eax, (%rbx)
jle    18e0 <phase_6+0xe8>
```

得到最后应该为升序排列，否则就会爆炸，将节点对应的值排序后得到5 3 1 4 6 2。

secret_phase

答案: ccaac

查阅资料得可以从phase_defused中得到秘密炸弹的信息，disas该函数后得到这行信息

```
0x000055555555615a <+131>:    lea    0x14e0(%rip),%rsi    # 0x555555557641
```

查找此地址处得到binary。

但是受AI影响以为该入口在phase4处，得到程序一直在循环运行

```
0x000055555555610f <+56>:    cmp    $0x20,%c1           ; 检查字符是否是空格 (0x20)
0x0000555555556112 <+59>:    sete   %c1                 ; 如果是空格, c1 = 1, 否则 c1 = 0
0x0000555555556115 <+62>:    movzbl %c1,%ecx
0x0000555555556118 <+65>:    add    %ecx,%edx           ; edx 计数器 +1 (统计空格数量?)
0x000055555555611a <+67>:    mov    %eax,%esi
0x000055555555611c <+69>:    movzbl (%rdi,%rax,1),%ecx ; 读取下一个字符
0x0000555555556120 <+73>:    add    $0x1,%rax          ; 指针 +1
0x0000555555556124 <+77>:    cmp    $0x5,%edx          ; 检查计数器是否 > 5
0x0000555555556127 <+80>:    jg     0x55555555612d      ; 如果 > 5, 跳出循环
0x0000555555556129 <+82>:    test   %c1,%c1            ; 检查字符是否是 '\0' (结束符)
0x000055555555612b <+84>:    jne    0x55555555610f      ; 如果不是结束符, 继续循环
```

由5个以上空格猜测为phase6后加上关键词，成功进入secret bomb

从偏移量为600处也可获得应该在phase6

```
Breakpoint 2, 0x000055555555610f in phase_defused ()
(gdb) x/s $rdi
0x55555555a978 <input_strings+600>:    "5 3 1 4 6 2"
```

我们先看到有大量的从-2到2的整数赋值，在结合后面初始为 (0, 0) 要到 (4, 7) 可得这应该是横纵坐标的移动量。

```
cmp    $0x4,%esi
jne    1aaa <func7+0x18e>
cmp    $0x7,%edx
jne    1aaa <func7+0x18e>
```

mov %esi,%r10d 和 and \$0x7,%r10d 说明是和0x7做了与运算，保留了后三位，是一个字符和操作的索引对应。

索引 (低3位)	对应字符示例	dx (X变化)	dy (Y变化)
0	h, p, x	-2	+1
1	a, i, q	-1	+2
2	b, j, r	+1	+2
3	c, k, s	+2	+1
4	d, l, t	+2	-1
5	e, m, u	+1	-2
6	f, n, v	-1	-2

接着需要找到地图，来确定路径。查找偏移量为0x61a0处，得到地图为

```
00100100
00010001
10100100
10000000
01001010
```

可试得路径 (0, 0) - (2, 1) - (4, 2) - (3, 4) - (2, 6) - (4, 7)

由表格可得应输入字符串为ccaac。

```
344
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
Sho fu briyu praffi stassui tsenva chies, Ien ryus sois nyat pyaro shennie fru.
Phase 1 defused. (Interesting key, huh?)
403295 618398 409883 587284
That's number 2.
5 -460
Halfway there!
Ancient monks moved sacred disks between poles...
31 BA
So you got that one. Try this one.
mmmmmm
Good work! On to the next...
5 3 1 4 6 2 binary
Curses, you've found the secret phase!
But finding it and solving it are quite different...
Wait! Is there a horse?
ccaac
Wow! The horse gallops to victory! You are a master!
Congratulations! You've defused the bomb!
Your instructor has been notified and will verify your solution.
[Inferior 1 (process 950) exited normally]
(gdb) █
```

反馈/收获/感悟/总结

汇编代码里很多东西一开始不知道作用是什么，其实很多到后面也不知道在干嘛，但能通过一些跳转条件能知道一些信息。感觉还是挺难的，没有那么好理解

参考的重要资料
